

TP2 — Pipeline data temps réel & plateforme data

Prolongement du TP1 — Immobilier × Énergie, Loire-Atlantique (44)

Louis Valette

Alexis Fouquet

Ruben Cofflard

Contents

1. Architecture	1
2. Sources	2
3. Data lake	2
4. Agrégation et nettoyage	3
5. Observabilité (Prometheus + Grafana)	3
6. Data visualization	3
7. Vérification	3

1. Architecture

Le TP1 a construit une base PostgreSQL statique (import unique, département 44) croisant DVF et DPE. Le TP2 ajoute un pipeline **temps réel** rechargeant la même base en continu : Kafka, data lake brut, Spark Structured Streaming, PostgreSQL, supervision et dashboard métier.

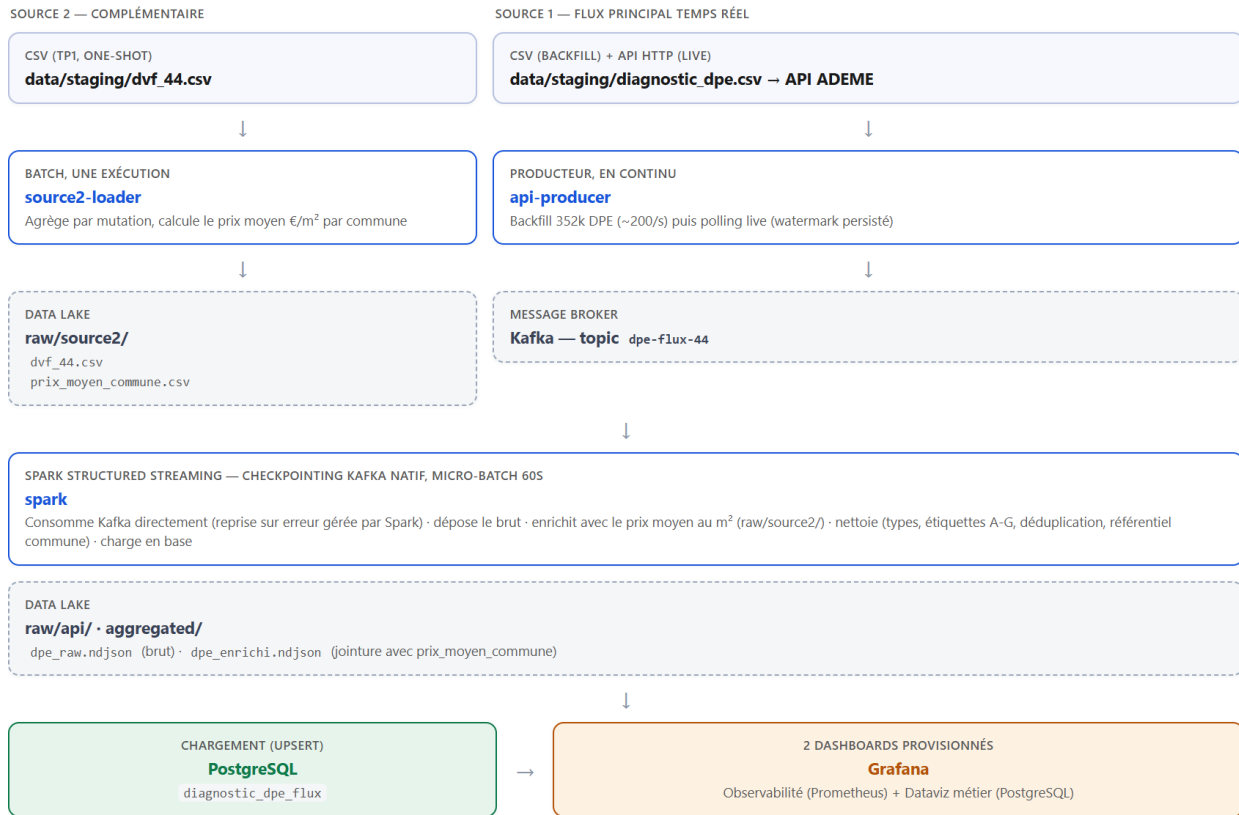


Figure 1: Architecture du pipeline temps réel TP2

2. Sources

#	Source	Nature	Rôle
1	API ADEME dpe03existant	API HTTP, alimentée sur Kafka par api-producer	Flux principal temps réel
2	DVF (département 44, déjà utilisée en TP1)	CSV, chargée en un lot par source2-loader	Enrichit le flux 1 (prix moyen au m² par commune)

L'API ADEME ne produit que quelques nouveaux DPE par semaine pour le seul département 44 — insuffisant pour une démo courte. api-producer rejoue donc d'abord les 352k DPE du TP1 à débit maîtrisé (200/s, source_flux=backfill), puis bascule en interrogation continue de l'API réelle (source_flux=live, watermark persisté). Le backfill ne s'exécute qu'une fois (/state/backfill_done).

3. Data lake

MinIO (S3, proposé par le sujet) a fermé ses images Docker officielles derrière une authentification (Docker Hub et quay.io renvoient 401), incompatible avec un docker compose up reproductible sans compte préalable. Le data lake est un volume Docker nommé, organisé selon l'arborescence de l'énoncé :

```

raw/api/          dpe_raw.ndjson          — flux Kafka brut, non filtré
raw/source2/      dvf_44.csv, prix_moyen_commune.csv

```

aggregated/ dpe_enrichi.ndjson – flux enrichi, avant nettoyage

4. Agrégation et nettoyage

spark/jobs/clean_and_load.py consomme le topic Kafka dpe-flux-44 directement en Spark Structured Streaming (micro-batch 60s, checkpointing Kafka natif). Par micro-batch :

1. dépôt du brut dans raw/api/ ;
2. **enrichissement** : jointure avec le prix moyen au m² de la commune (calculé par source2-loader, même correction qu'en TP1 : agrégation par mutation avant calcul du prix/m²) → aggregated/ ;
3. **nettoyage** : types, étiquettes DPE/GES dans A-G, déduplication sur numero_dpe, rejet des communes hors référentiel (FK vers commune) ;
4. upsert dans diagnostic_dpe_flux ([sql/06_create_flux_table.sql](#), ON CONFLICT DO NOTHING).

Le connecteur Kafka de Spark est résolu et figé dans l'image au build, pas téléchargé au démarrage : docker compose up -d ne dépend d'aucun accès réseau à l'exécution.

5. Observabilité (Prometheus + Grafana)

Cible	Exportateur	Expose
Conteneurs	cAdvisor	CPU/mémoire/réseau — remplace node-exporter (un seul hôte, pas de cluster)
PostgreSQL api-producer, spark	postgres-exporter prometheus_client	Connexions, disponibilité, taille Métriques métier du pipeline

Indicateur Raw vs Clean (obligatoire) : raw_records_total (volume déposé dans raw/api/) vs clean_records_total (COUNT(*) sur diagnostic_dpe_flux), tous deux recalculés depuis le volume persisté (pas un compteur remis à zéro au redémarrage). raw ≥ clean en permanence.

Dashboard *TP2 — Pipeline temps réel* provisionné automatiquement : 8 panels (Raw vs Clean, taux de nettoyage, débit du producteur, PostgreSQL, CPU/mémoire).

6. Data visualization

Le sujet demande un dashboard sans imposer d'outil. Plutôt qu'un second outil dédié (Metabase), la dataviz métier est un second dashboard Grafana *TP2 — Dataviz métier*, sur un second datasource PostgreSQL : répartition des étiquettes DPE, origine backfill/live, volume dans le temps, communes les plus chères, part F-G par tranche de prix. Provisionné comme le dashboard technique, sans étape manuelle — contrairement à un outil séparé dont le dashboard se construit à la main.

7. Vérification

Stack testée à froid (docker compose down -v && docker compose up -d) sur trois machines et réseaux différents, y compris reprise après un docker kill de Spark en plein flux (checkpoint Kafka : ni perte ni doublon).

Étape	Résultat
Kafka (dpe-flux-44)	Backfill complet : 351 959 événements publiés
PostgreSQL (diagnostic_dpe_flux)	351 959 lignes après backfill complet
Raw vs Clean	Vérifié à la source (fichiers + base) : raw ≥ clean à tout instant
Prometheus	4/4 cibles UP
Grafana	2 dashboards + 2 datasources provisionnés ; requêtes vérifiées avec données réelles

Le classement des communes les plus chères, recalculé en continu, retombe sur les valeurs du TP1 (La Baule-Escoublac 6 536 €/m², Pornichet 5 636 €/m², Nantes 3 777 €/m²).

Comment reproduire

```
docker compose up -d
```

Démarré l'ensemble (TP1 + TP2) sans étape manuelle ni accès réseau obligatoire (le backfill utilise les CSV déjà versionnés).

Comment vérifier

Composant	Commande
Kafka	<code>docker exec tp2_kafka /opt/kafka/bin/kafka-get-offsets.sh --bootstrap-server localhost:19092 --topic dpe-flux-44</code>
Data lake	<code>docker exec tp2_spark sh -c "wc -l /datalake/raw/api/dpe_raw.ndjson /datalake/aggregated/dpe_enrichi.ndjson"</code>
PostgreSQL	<code>docker exec tp_audit_44_pg psql -U postgres -d audit_immo_energie_44 -c "SELECT COUNT(*) FROM diagnostic_dpe_flux;"</code>
Prometheus	<code>http://localhost:9090/targets</code>
Grafana	<code>http://localhost:3000 (admin/admin)</code>

Ports exposés

Port	Service
5432	PostgreSQL
9092	Kafka (accès externe)
8001 / 8003	Métriques Prometheus : producteur / Spark
8080	cAdvisor
9187	postgres-exporter
9090	Prometheus
3000	Grafana